Lab5：时序逻辑电路 实验报告

PB24111621 肖瀚

一、实验目的与内容

1. 学习了解基本的时序逻辑元件，如锁存器、触发器、寄存器等；
2. 学习有限状态机(FSM)的相关概念以及如何用三段式代码实现 FSM；
3. 学会运用 FSM 去解决一些时序逻辑电路的问题。

二、逻辑设计

【必做部分】

题目1：寄存器堆

先将所有的寄存器初始化为 0，再根据写使能信号和写端口地址判断是否需要写入，最后读出端口 1 和 2 的数据。这里实现同步读写的方式是：在读端口部分的 `if-else` 语句中，执行读取写入数据 `dout1 = din` 的操作比执行读操作 `dout1 = reg_file[ra1]` 的判断条件更靠前，也就是先判断是否读取写端口的数据，若是，则读取写端口的数据，若不是，则读取非写端口的数据。解答代码如下：

```
module Regfile (
    input                       clk,          // 时钟信号
    input           [4:0]       ra1,          // 读端口 1 地址
    input           [4:0]       ra2,          // 读端口 2 地址
    input           [4:0]       wa,           // 写端口地址
    input                       we,           // 写使能信号
    input           [31:0]      din,          // 写数据
    output  reg     [31:0]      dout1,        // 读端口 1 数据输出
    output  reg     [31:0]      dout2         // 读端口 2 数据输出
);

reg [31:0] reg_file [31:0];
integer i;

// 初始化
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        reg_file[i] = 32'b0;
    end
end

always @(posedge clk) begin
    if (we && (wa != 5'b0)) begin
        reg_file[wa] <= din;
    end
end

// 读端口1
```

```
always @(*) begin
    if (ra1 == 5'b0) begin
        dout1 = 32'b0;
    end
    else if (we && (ra1 == wa) && (wa != 5'b0)) begin
        dout1 = din;
    end
    else begin
        dout1 = reg_file[ra1];
    end
end

// 读端口2
always @(*) begin
    if (ra2 == 5'b0) begin
        dout2 = 32'b0;
    end
    else if (we && (ra2 == wa) && (wa != 5'b0)) begin
        dout2 = din;
    end
    else begin
        dout2 = reg_file[ra2];
    end
end

endmodule
```

题目2：数码管时钟

由于不允许使用 % 运算符，故在代码中，采用多个 if-else 语句来实现进位的操作，具体而言，就是判断秒钟个位是否是 9，如果是，则个位变为 0，再判断秒钟十位是否需要进位，以此类推；如果不是，则秒钟个位加 1，其他位不做修改。

另外，在代码中添加了一个分频计数器，每计满一秒就会有一个简短的标记信号 clk1s = 1，从而进行后面时钟的变化操作。

具体的解答代码如下：

```
module TIMER (
    input                       clk, rst,
    output          [3:0]       out,
    output          [2:0]       select
);

    reg [3:0]  outh ;
    reg [3:0]  outms;
    reg [3:0]  outmg;
    reg [3:0]  outss;
    reg [3:0]  outsg;
    reg [39:0] count;
    wire clk1s;
```

```
    initial begin
        outh    = 4'H9 ;
        outms   = 4'H5 ;
        outmg   = 4'H8 ;
        outss   = 4'H3 ;
        outsg   = 4'H0 ;
        count   = 40'H0;
    end

    always @(posedge clk) begin
        if (rst)
            count <= 40'd0;
        else begin
            if (count == 40'd9999_9999)
                count <= 40'd0;
            else
                count <= count + 40'd1;
        end
    end

    assign clk1s = (count == 40'd49_999_999);

    Segment segment(
        .clk                (clk),
        .btn                (rst),
        .output_data        ({outh, outms, outmg, outss, outsg}),
        .seg_data           (out),
        .seg_an             (select)
    );

    always @(posedge clk) begin
        if(rst) begin
            outh  <= 4'H9;
            outms <= 4'H5;
            outmg <= 4'H8;
            outss <= 4'H3;
            outsg <= 4'H0;
        end
        else if(clk1s) begin
            if(outsg == 9) begin
                outsg <= 0;
                if(outss == 5) begin
                    outss <= 0;
                    if(outmg == 9) begin
                        outmg <= 0;
                        if(outms == 5) begin
                            outms <= 0;
                            if(outh == 4'HF)
                                outh <= 0;
                            else
                                outh <= outh + 1;
                        end
                        else
```

```
                            outms <= outms + 1;
                    end
                    else
                        outmg <= outmg + 1;
                end
                else
                    outss <= outss + 1;
            end
            else
                outsg <= outsg + 1;
        end
    end

endmodule
```

【选择性必做内容】

题目2：投币机

首先定义状态变量以及状态名称：

```
reg  current_state, next_state;
localparam s0 = 1'd0;
localparam s1 = 1'd1;
reg Y_reg, Z_reg;
```

接下来编写第一段：状态更新。 `rst` 信号能将状态复位到 s0

```
always @(posedge clk) begin
    if (rst)
        current_state <= s0;
    else
        current_state <= next_state;
end
```

接下来编写第二段：状态转移。根据状态转换图，可以编写如下的代码：

```
always @(posedge clk) begin
    next_state <= current_state;
    Y_reg <= (~A & B) | (A & ~B & current_state);
    Z_reg <= (~A & B & current_state);
    case (current_state)
        s0: begin
            if(A && ~B)
                next_state <= s1;
            else
                next_state <= s0;
```

```
        end

        s1: begin
            if(~A && ~B)
                next_state <= s1;
            else
                next_state <= s0;
        end
        default: next_state <= s0;
    endcase
end
```

最后，编写第三段：输出。

```
assign Y = Y_reg;
assign Z = Z_reg;
```

完整代码如下：

```
module CoinMachine (
    input    clk,
    input    rst,
    input    A,          // 投入5分
    input    B,          // 投入10分
    output   Y,          // 出饮料
    output   Z           // 找零
);

reg  current_state, next_state;
localparam s0 = 1'd0;
localparam s1 = 1'd1;
reg Y_reg, Z_reg;

// 第一段：状态更新
always @(posedge clk) begin
    if (rst)
        current_state <= s0;
    else
        current_state <= next_state;
end

// 第二段：状态转移
always @(posedge clk) begin
    next_state <= current_state;
    Y_reg <= (~A & B) | (A & ~B & current_state);
    Z_reg <= (~A & B & current_state);
    case (current_state)
        s0: begin
            if(A && ~B)
```

```
                next_state <= s1;
            else
                next_state <= s0;
        end

        s1: begin
            if(~A && ~B)
                next_state <= s1;
            else
                next_state <= s0;
        end
        default: next_state <= s0;
    endcase
end

// 第三段：输出
assign Y = Y_reg;
assign Z = Z_reg;

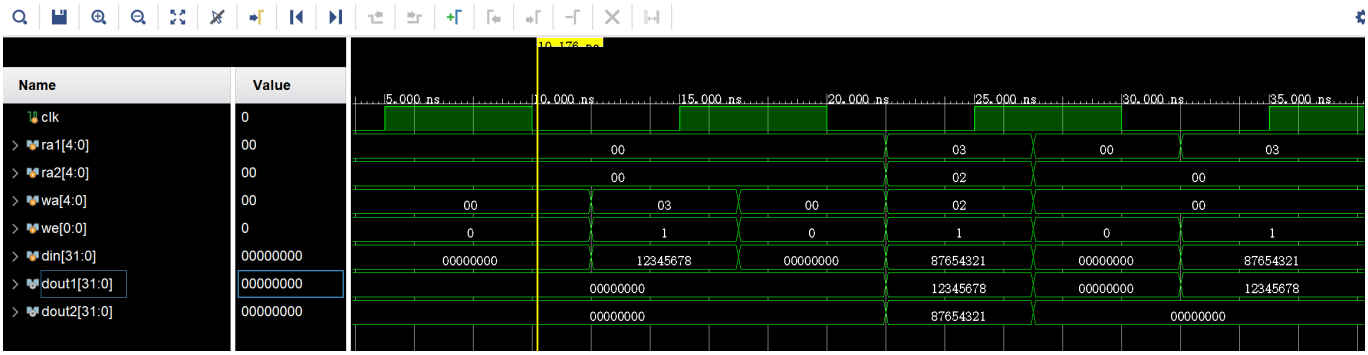
endmodule
```

三、仿真与测试结果分析

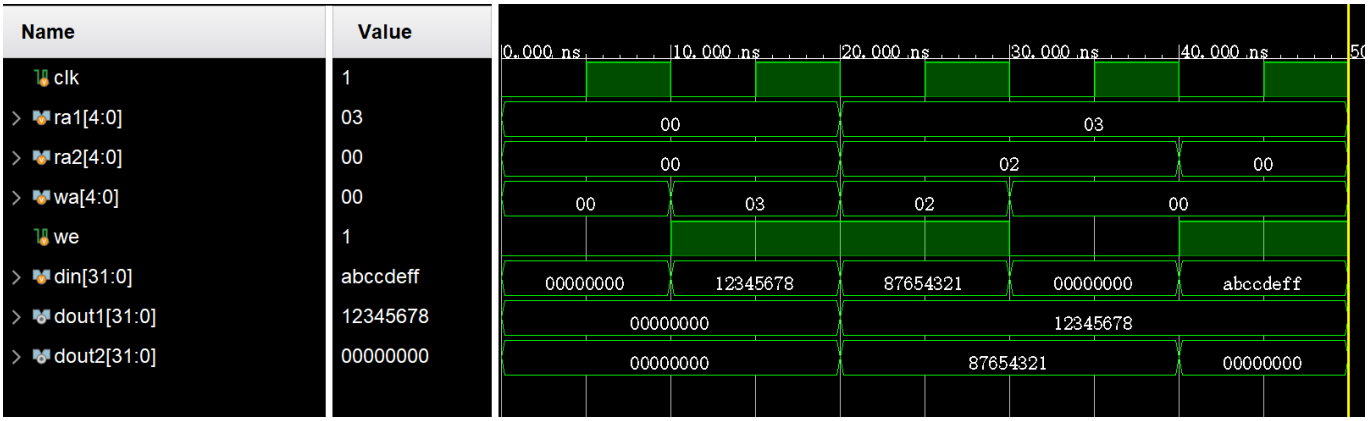
【必做内容】

题目1的仿真

利用助教给的仿真文件，得到的仿真结果如下，这与实验教程中的贴图是一致的



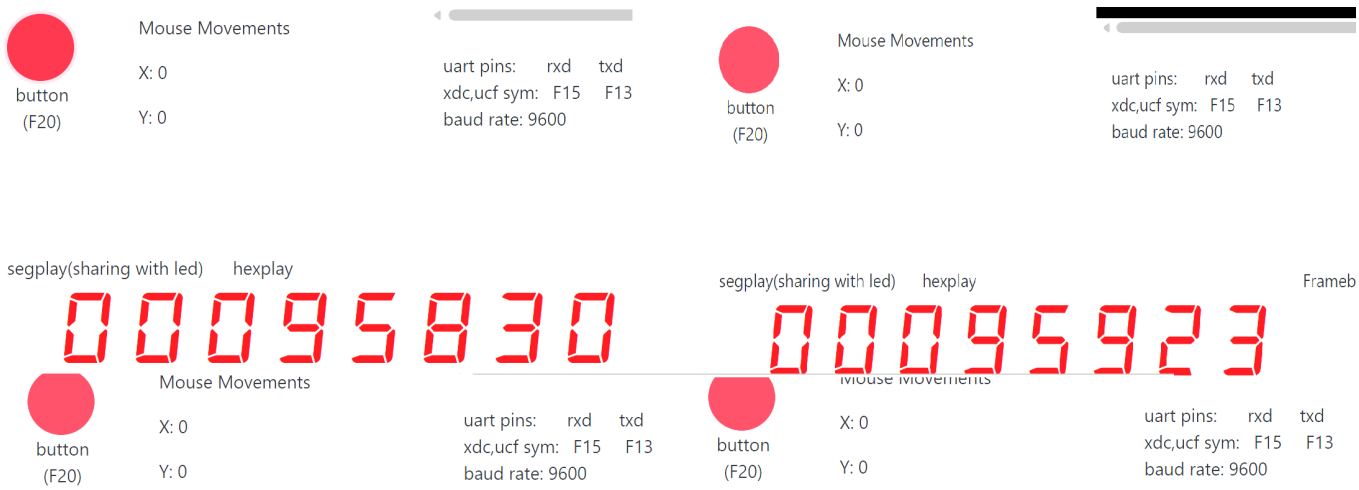
利用自己编写的仿真文件(已附入提交的zip文件中)得到的波形图如下。



结果分析：

- 当写入地址为03时，写入12345678，由于此时读的端口均为 00 ，故输出的两个数据全为 0
- 当写入地址为02时，写入87654321，此时端口 1 的输出为地址 03 的12345678，端口 2 读取位置的地址为 02 ，由于写优先，故读取的是写入的数据87654321
- 当写入地址为00时，不写入数据，此时端口 2 读取的数据是 00000000

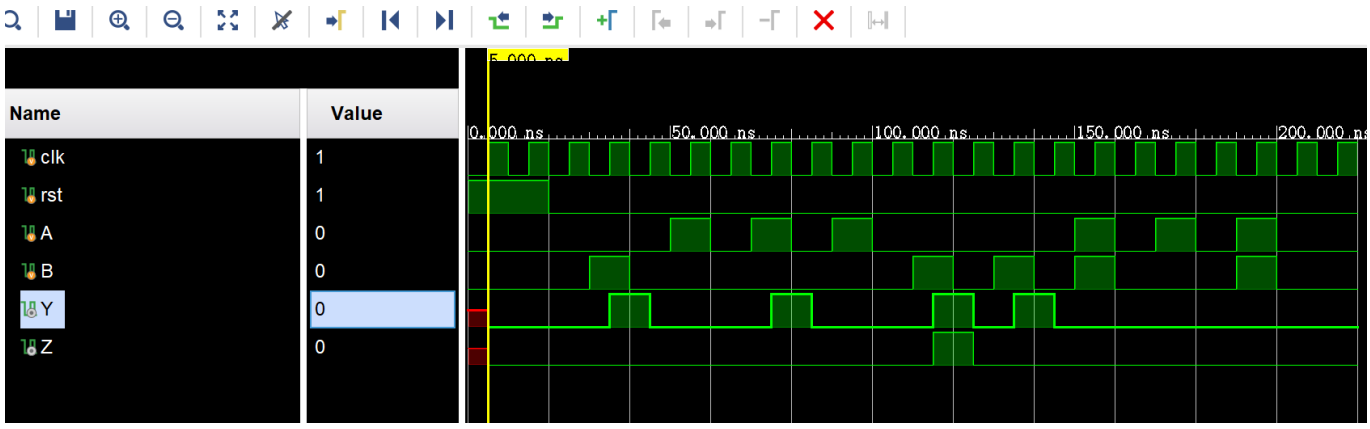
题目2的上板结果





【选择性必做部分】

题目2的仿真



结果分析：

- 第一次投入10，只出饮料，不找零，YZ = 10，保持状态 s0
- 投入5，不出饮料也不找零，YZ = 00，变为状态s1
- 投入5，出饮料不找零，YZ = 10，转为s0
- 投入5，不出饮料也不找零，YZ = 00，变为状态s1
- 投入10，出饮料并找零，YZ = 11，回到s0

- 投入10，出饮料不找零，保持s0
- 投入5和10，非法投入，YZ = 00，保持状态 s0
- 投入5，不出饮料也不找零，YZ = 00，变为状态s1
- 投入5，出饮料不找零，YZ = 10，转为s0

四、总结

通过本次实验，我对时序逻辑电路有了初步了解，认识了锁存器、寄存器等时序电路元件，也学习了有限状态机(FSM)，学会如何用 Verilog 三段式代码表示一个有限状态机，并利用有限状态机去解决简单的实际问题。